

Data 236- Distributed Systems

Homework-7

Kruthika Virupakshappa

Part 1-Blog Management System

Build a small FastAPI backend for a Blog Management System that demonstrates:

- JWT-based authentication
- Role-based authorization
- Linting with Ruff and pre-commit hooks

create Blog Management System with these roles:

- reader
- writer
- moderator

Functional Requirements:

Authentication:

- successful login returning JWT token

```
router = APIRouter(prefix="/auth", tags=["Authentication"])

@router.post("/login", response_model=Token)
def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db),
):
    user = db.query(User).filter(User.username == form_data.username).first()

    if not user or not verify_password(form_data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    access_token = create_access_token(
        data={"sub": user.username, "role": user.role}
    )

    return {"access_token": access_token, "token_type": "bearer"}
```

Authentication

POST

/auth/login

Login

Parameters

Cancel

Reset

No parameters

Request body

required

application/x-www-form-urlencoded

grant_type

string | (string | null)

pattern: password

password

☐ Send empty value

username

required

string

alice

password

required

string

alice123

scope

string

scope

☒ Send empty value

client_id

string | (string | null)

string

☐ Send empty value

client_secret

string | (string | null)

string

☐ Send empty value

Execute

Clear

Responses

Curl

curl -X 'POST' \ 'http://127.0.0.1:8001/auth/login' \ -H 'accept: application/json' \ -H 'Content-Type: application/x-www-form-urlencoded' \ -d 'grant_type=password&username=alice&password=alice123&scope=&client_id=string&client_secret=string'

Request URL

http://127.0.0.1:8001/auth/login

Server response

Responses		
Curl curl -X 'POST' \ 'http://127.0.0.1:8001/auth/login' \ -H 'accept: application/json' \ -H 'Content-Type: application/x-www-form-urlencoded' \ -d 'grant_type=password&username=alice&password=alice123&scope=&client_id=string&client_secret=string' Request URL http://127.0.0.1:8001/auth/login Server response		
Code	Details	
200	Response body { "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1bGU1LjY2ZWZkZXIiLCJleHAiOiJlbnQyMjQSMOR9.UF61zab-YYK5-vTvsC1FesVA3f1Y8-auhpFjWq65o", "token_type": "bearer" } Response headers content-length: 186 content-type: application/json date: Sun, 22 Mar 2026 23:45:04 GMT server: uvicorn	Download
Responses		
Code	Description	Links
200	Successful Response	No links
Media type application/json Controls Accept header. Example Value Schema { "access_token": "string", "token_type": "string" }		

- failed login returning 401 Unauthorized

```
@router.post("/login", response_model=Token)
def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db),
):
    user = db.query(User).filter(User.username == form_data.username).first()

    if not user or not verify_password(form_data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    access_token = create_access_token(
        data={"sub": user.username, "role": user.role}
    )

    return {"access_token": access_token, "token_type": "bearer"}
```

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=password&username=alice&password=aswasa&scope=&client_id=string&client_secret=string'
```

Request URL

```
http://127.0.0.1:8001/auth/login
```

Server response

Code	Details
401	Error: Unauthorized

Undocumented

Response body

```
{
  "detail": "Incorrect username or password"
}
```

Response headers

```
content-length: 43
content-type: application/json
date: Sun, 22 Mar 2026 23:47:55 GMT
server: uvicorn
www-authenticate: Bearer
```

Responses

Authorization:

- GET /users/me without token returning 401 Unauthorized

Users

GET /users/me Read Current User

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8001/users/me' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8001/users/me
```

Server response

Code	Details
401	Error: Unauthorized

Undocumented

Response body

```
{
  "detail": "Not authenticated"
}
```

Response headers

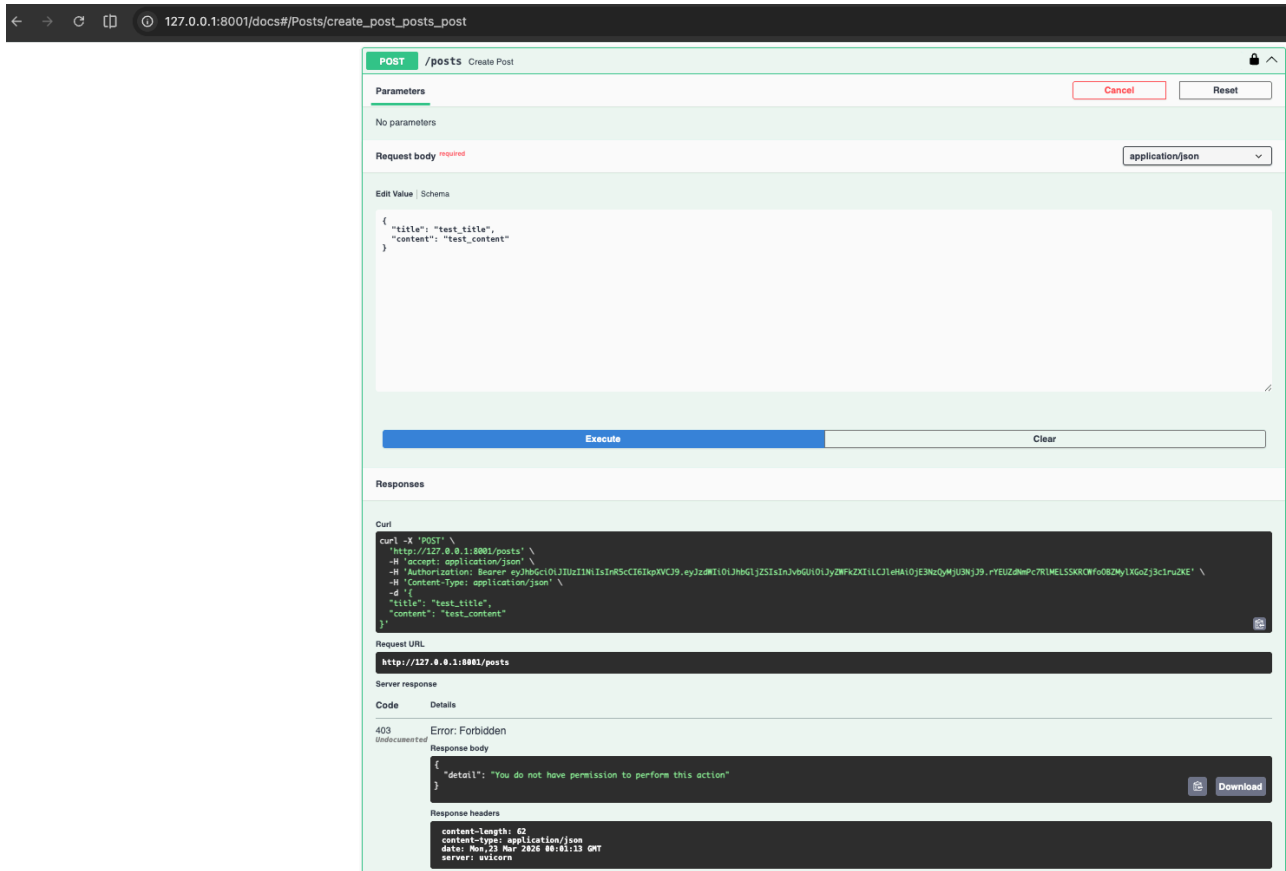
```
content-length: 30
content-type: application/json
date: Sun, 22 Mar 2026 23:58:13 GMT
server: uvicorn
www-authenticate: Bearer
```

Responses

- POST /posts using reader role returning 403 Forbidden

```
def require_roles(allowed_roles: list[str]) -> Callable:
    def role_checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="You do not have permission to perform this action",
            )
        return current_user
```

```
@router.post("", response_model=PostResponse, status_code=status.HTTP_201_CREATED)
def create_post(
    post: PostCreate,
    current_user: User = Depends(require_roles(["writer", "moderator"])),
    db: Session = Depends(get_db),
):
    new_post = Post(
        title=post.title,
        content=post.content,
        author=current_user.username,
    )
    db.add(new_post)
    db.commit()
    db.refresh(new_post)
    return new_post
```

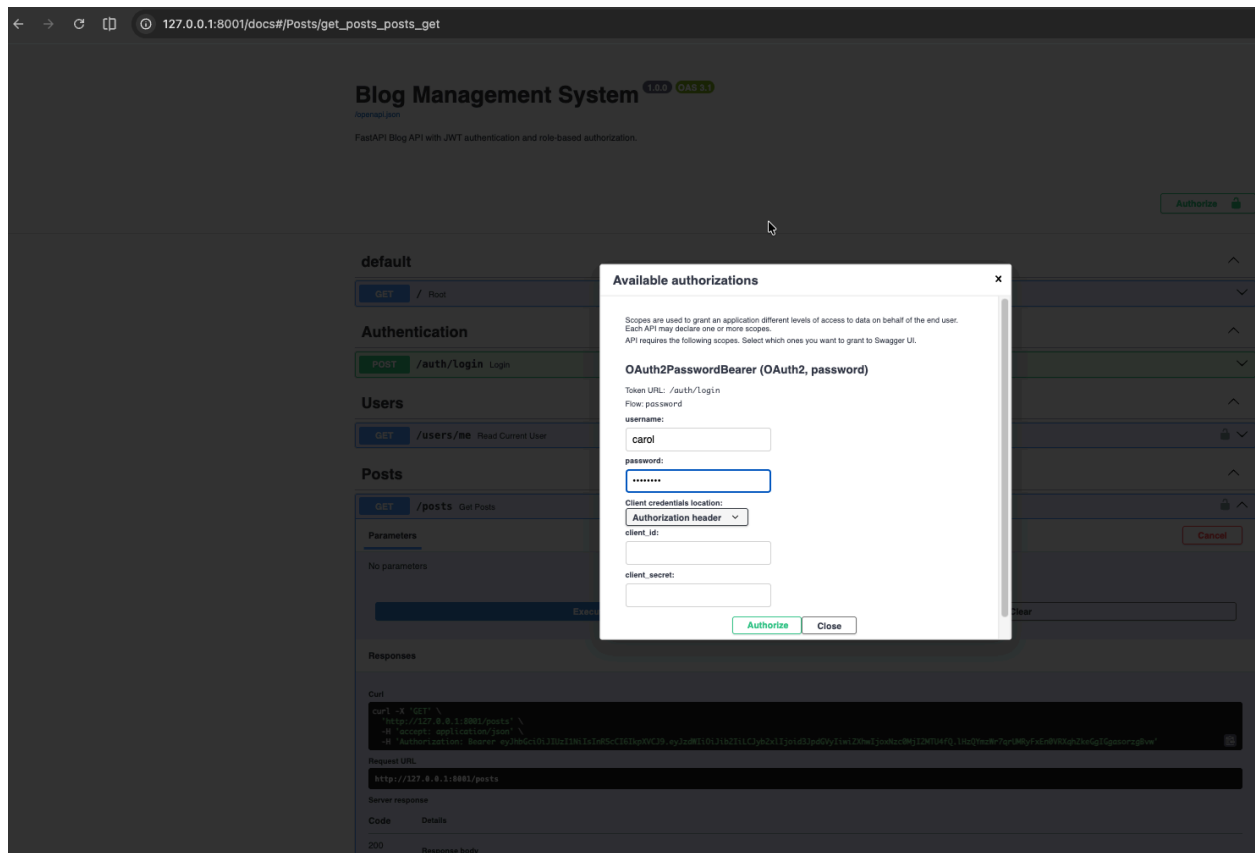


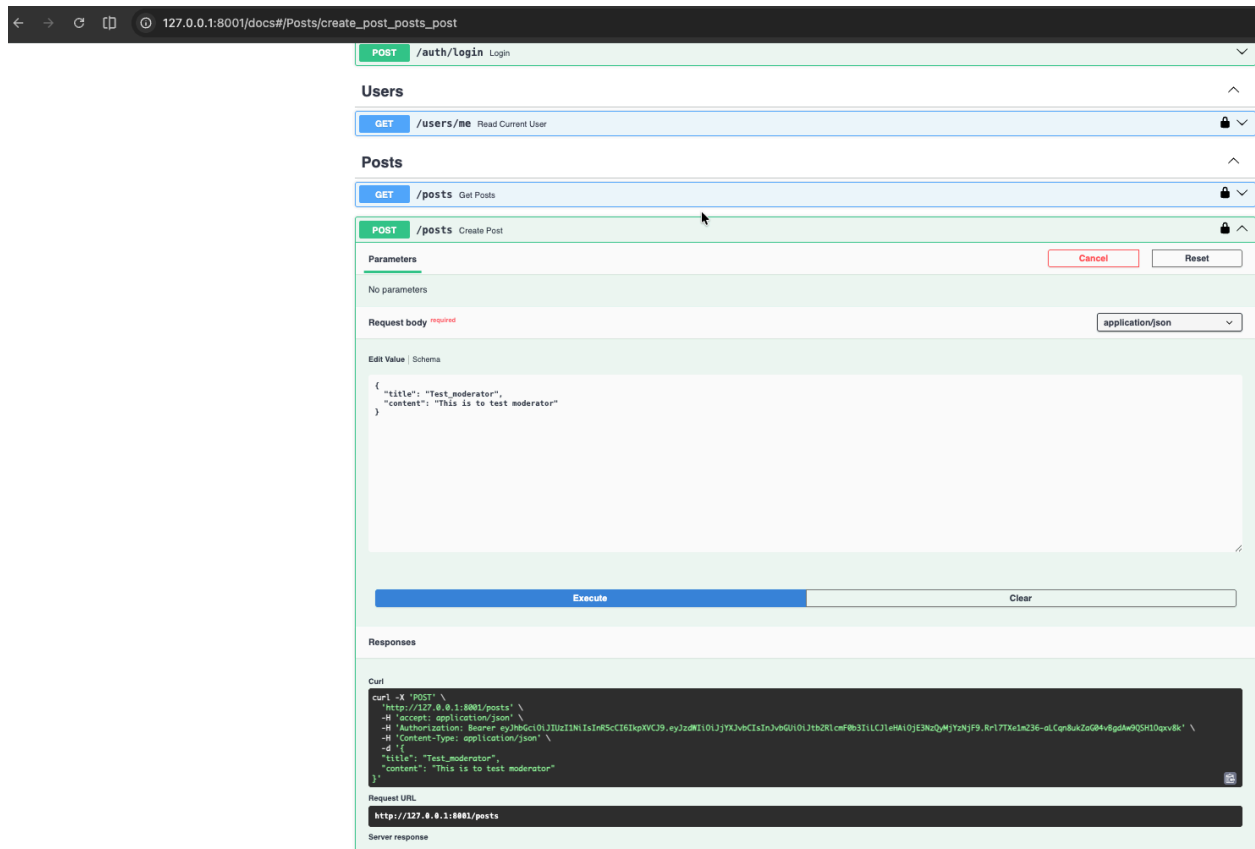
• POST /posts using writer or moderator returning 201 Created

```
@router.post("", response_model=PostResponse, status_code=status.HTTP_201_CREATED)
def create_post(
    post: PostCreate,
    current_user: User = Depends(require_roles(["writer", "moderator"])),
    db: Session = Depends(get_db),
):
    new_post = Post(
        title=post.title,
        content=post.content,
        author=current_user.username,
    )
    db.add(new_post)
    db.commit()
    db.refresh(new_post)
    return new_post
```

Writer Authorization

Moderator login:





- DELETE /posts/{id} using writer returning 403 Forbidden

```
def require_roles(allowed_roles: list[str]) -> Callable:
    def role_checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="You do not have permission to perform this action",
            )
        return current_user

    return role_checker
```

```
@router.delete("/{post_id}")
def delete_post(
    post_id: int,
    current_user: User = Depends(require_roles(["moderator"])),
    db: Session = Depends(get_db),
):
    post = db.query(Post).filter(Post.id == post_id).first()
    if not post:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Post not found",
        )
    db.delete(post)
    db.commit()
    return {
        "message": "Post deleted successfully",
        "deleted_post_id": post_id,
        "deleted_by": current_user.username,
    }
```


The screenshot shows a REST client interface with the following details:

- Method:** DELETE
- URL:** /posts/{post_id} Delete Post
- Parameters:**
 - post_id:** integer (path), value: 1
- Execute:** Button to execute the request.
- Responses:**
 - Code:** 200
 - Response body:**

```
{
  "message": "Post deleted successfully",
  "deleted_post_id": 1,
  "deleted_by": "carol"
}
```
 - Response headers:**

```
content-length: 88
content-type: application/json
date: Mon, 23 Mar 2026 08:18:21 GMT
server: uvicorn
```
- Links:** No links

- GET /posts using valid token returning 200 OK

```
@router.get("", response_model=list[PostResponse])
def get_posts(
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    return db.query(Post).all()
```

← → ↻ 127.0.0.1:8001/docs#/Posts/get_posts_posts_get

GET /users/me Read Current User

Posts

GET /posts Get Posts

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8001/posts' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1b3R5b2VhZCIsInVzZ21ka2kiOiJ1b3R5b2VhZCIsImF1dG8iOiJlYXNpdG93aW50IiwiaWF0IjoxNjU0MjQ0MDA0LCJpc29udG8iOiJlYXNpdG93aW50In0'>
```

Request URL

http://127.0.0.1:8001/posts

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "title": "Test_moderator",
  "content": "This is to test moderator",
  "author": "carol",
  "created_at": "2025-03-22T17:09:48"
}
```

Response headers

```
content-length: 125
content-type: application/json
date: Mon, 23 Mar 2026 08:16:42 GMT
server: uvicorn
```

Responses

← → ↻ 127.0.0.1:8001/docs#/Posts/get_posts_posts_get

Authentication

POST /auth/login Login

Users

GET /users/me Read Current User

Posts

GET /posts Get Posts

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8001/posts' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1b3R5b2VhZCIsInVzZ21ka2kiOiJ1b3R5b2VhZCIsImF1dG8iOiJlYXNpdG93aW50IiwiaWF0IjoxNjU0MjQ0MDA0LCJpc29udG8iOiJlYXNpdG93aW50In0'>
```

Request URL

http://127.0.0.1:8001/posts

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "title": "Test_Writer",
  "content": "This is to test writer",
  "author": "bob",
  "created_at": "2025-03-22T17:06:36"
}
```

Response headers

```
content-length: 117
content-type: application/json
date: Mon, 23 Mar 2026 08:16:00 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header

Linting / Pre-commit

- terminal screenshot showing linting error before commit

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  SPELL CHECKER  GITLENS

1 | / import os
2 | import math
3 | from fastapi import APIRouter, Depends, HTTPException, status
4 | from sqlalchemy.orm import Session
5 |
6 | from app.core.dependencies import get_current_user, require_roles
7 | from app.db.database import get_db
8 | from app.models.post import Post
9 | from app.models.user import User
10 | from app.schemas.post import PostCreate, PostResponse
11 |
12 | router = APIRouter(prefix="/posts", tags=["Posts"])
    ^ I001
    = help: Organize imports

app/routers/posts.py:1:8: F401 [*] `os` imported but unused
1 | import os
    ^^ F401
2 | import math
3 | from fastapi import APIRouter, Depends, HTTPException, status
    = help: Remove unused import: `os`

app/routers/posts.py:2:8: F401 [*] `math` imported but unused
1 | import os
2 | import math
    ^^^ F401
3 | from fastapi import APIRouter, Depends, HTTPException, status
4 | from sqlalchemy.orm import Session
    = help: Remove unused import: `math`

Found 3 errors.
[*] 3 fixable with the `--fix` option.
```

- terminal screenshot showing 1 or 2 issues fixed

```
(env) Kruthika@Kruthikas-MacBook-Air blog_management % git add .
(env) Kruthika@Kruthikas-MacBook-Air blog_management % git commit -m "add blog management system"
ruff.....Failed
- hook id: ruff
- files were modified by this hook

Found 2 errors (2 fixed, 0 remaining).

ruff-format.....Passed
(env) Kruthika@Kruthikas-MacBook-Air blog_management % ruff check --fix app/
All checks passed!
(env) Kruthika@Kruthikas-MacBook-Air blog_management %
```

- terminal screenshot showing successful commit

```

• (env) Kruthika@Kruthikas-MacBook-Air blog_management % ruff check --fix app/
All checks passed!
• (env) Kruthika@Kruthikas-MacBook-Air blog_management % git add .
• (env) Kruthika@Kruthikas-MacBook-Air blog_management % git commit -m "add blog management app"
ruff..... Passed
ruff-format..... Passed
[main (root-commit) 048fdf8] add blog management app
24 files changed, 422 insertions(+)
create mode 100644 .gitignore
create mode 100644 .pre-commit-config.yaml
create mode 100644 app/__init__.py
create mode 100644 app/core/__init__.py
create mode 100644 app/core/config.py
create mode 100644 app/core/dependencies.py
create mode 100644 app/core/security.py
create mode 100644 app/db/__init__.py
create mode 100644 app/db/database.py
create mode 100644 app/main.py
create mode 100644 app/models/__init__.py
create mode 100644 app/models/post.py
create mode 100644 app/models/user.py
create mode 100644 app/routers/__init__.py
create mode 100644 app/routers/auth.py
create mode 100644 app/routers/posts.py
create mode 100644 app/routers/users.py
create mode 100644 app/schemas/__init__.py
create mode 100644 app/schemas/auth.py
create mode 100644 app/schemas/post.py
create mode 100644 app/schemas/user.py
create mode 100644 pyproject.toml
create mode 100644 requirements.txt
create mode 100644 seed_db.py
○ (env) Kruthika@Kruthikas-MacBook-Air blog_management %

```

Part 2: Single-Agent ReAct + MRKL Bike-Share Pass Optimizer (Public Data)

Chosen City : New York City — Citi Bike

Deliverables

- A small web UI that:
 - Accepts: a **month** CSV from the chosen city (user uploads one file) and a **URL** to the official pricing page.
 - Shows a **timeline** of agent thinking process or delegation of tasks to tools.
 - Shows a **cost comparison**: pay-per-ride/minute vs monthly membership, with any **overage/minute surcharges** and **included ride durations** applied per policy.
 - Includes **citations** to the pricing/policy dataset sections you used.
- A **single agent** using a **ReAct loop** + **MRKL tools**.
- Per-step logs: tool, args hash, latency, success/failure, stop reason.

```

client = AsyncOpenAI(
    api_key=os.environ["GROQ_API_KEY"],
    base_url="https://api.groq.com/openai/v1",
)

SYSTEM_PROMPT = """You are a bike-share financial advisor. Analyze a rider's Citi Bike trip CSV and recommend Monthly Membership or Pay Per Ride/Minute.

Tools available:

```

- csv_sql: run SQL on the DuckDB table named `trips`
- policy_retriever: fetch pricing page snippets
- calculator: evaluate arithmetic (digits and + - * / () . only)

Steps to follow in order:

1. Run `SELECT \* FROM trips LIMIT 3` to discover column names.
2. Run one query: COUNT(*) AS total_rides, AVG(DATEDIFF('minute', started_at, ended_at)) AS avg_duration_min, eBike percentage.
3. Weekly breakdown: WITH w AS (SELECT *, NTILE(4) OVER (ORDER BY started_at) AS week FROM trips) SELECT week, COUNT(*) AS rides, AVG(DATEDIFF('minute', started_at, ended_at)) AS avg_duration_min FROM w GROUP BY week ORDER BY week.
4. Fetch pricing policy to get unlock fee, monthly membership cost, included minutes, overage rates.
5. calculator: pay_per_use_total = total_rides * unlock_fee
6. calculator: break_even = membership_cost / unlock_fee
7. calculator: savings = pay_per_use_total - membership_cost

IMPORTANT rules:

- total_rides is the COUNT of rows, NOT avg_duration_min. These are different numbers.
- Monthly membership fee is a flat monthly cost. Do NOT multiply it by days.
- Use the exact total_rides from the CSV row count in all calculations.

Final answer must use EXACTLY these section headers:

****DECISION:**** Buy Monthly Membership OR Pay Per Ride/Minute

****JUSTIFICATION:****

3-6 sentences citing specific policy prices.

****COST BREAKDOWN:****

- Total rides analysed: N
- Pay-per-use total: \$X.XX
- Monthly membership: \$X.XX
- Break-even rides: N rides/month (actual: N)
- Savings: \$X.XX with [recommended plan]

****WEEKLY TABLE:****

Week	Rides	Avg Duration (min)	eBike %	Spend (pay-per-use \$)
-----	-----	-----	-----	-----

****ASSUMPTIONS & CAVEATS:****

- bullet list

****CITATIONS:****

- "[exact quote]" -- source: <URL>, captured: <date>

"""

Tools:

```
TOOLS = [
  {
    "type": "function",
    "function": {
      "name": "csv_sql",
      "description": "Run a read-only SQL SELECT query on the uploaded Citi Bike trip data. The DuckDB table is named 'trips'.",
      "parameters": {
        "type": "object",
        "properties": {
          "sql": {
            "type": "string",
            "description": "A SQL SELECT or WITH...SELECT query against the 'trips' table."
          }
        },
        "required": ["sql"]
      }
    }
  },
  {
    "type": "function",
    "function": {
      "name": "policy_retriever",
      "description": "Fetch the official pricing page and return quotable snippets about membership price, unlock fees, included minutes, and per-minute surcharges.",
      "parameters": {
        "type": "object",
        "properties": {

```

```

        "url": {"type": "string", "description": "URL of the pricing page."},
        "query": {"type": "string", "description": "What pricing info to extract."},
        "k": {"type": "integer", "description": "Max snippets to return (default 8)."},
    },
    "required": ["url", "query"],
},
},
},
{
    "type": "function",
    "function": {
        "name": "calculator",
        "description": "Evaluate a safe arithmetic expression. Only digits and + - * / ( ) . are allowed.",
        "parameters": {
            "type": "object",
            "properties": {
                "expression": {"type": "string", "description": "Arithmetic expression "},
                "units": {"type": "string", "description": "Optional label e.g. 'USD'."},
            },
        },
        "required": ["expression"],
    },
},
},
],
MAX_STEPS = 20

```

```

def _run_tool(name: str, args: dict, db_conn: duckdb.DuckDBPyConnection) -> dict:
    if name == "csv_sql":
        return csv_sql(args.get("sql", ""), db_conn)
    if name == "policy_retriever":
        return policy_retriever(args.get("url", ""), args.get("query", ""), args.get("k", 8))
    if name == "calculator":
        return calculator(args.get("expression", ""), args.get("units"))
    return {"success": False, "error": f"Unknown tool: {name}"}

```

1. Scenario where membership wins

CONFIGURE ANALYSIS

Monthly Trip CSV

Drop CSV here or click to browse

JC-M202602-citibike-tripdata.csv

Pricing Page URL

https://citibikenyc.com/pricing

LLM Model

Llama 3.3 70B (Groq)

Run Analysis

RUN METRICS

2

STEPS

3.95s

TOTAL TIME

final answer

STOP REASON

AGENT TIMELINE

THOUGHT #2

DECISION: Buy Monthly Membership

JUSTIFICATION:

The total rides analyzed is 539. The pay-per-use total is \$1617.00. The monthly membership costs \$14.95 per month, which translates to \$179.40 per year. The break-even point is approximately 5 rides per month. Since the rider has already exceeded this threshold, buying a monthly membership would be the more cost-effective option.

COST BREAKDOWN:

- Total rides analysed: 539

- Pay-per-use total: \$1617.00

- Monthly membership: \$14.95

- Break-even rides: 5 rides/month (actual: 539)

- Savings: \$1437.60 with Monthly Membership

WEEKLY TABLE:

	Week	Rides	Avg Duration (min)	eBike %	Spend (pay-per-use \$)
1	1	135	19.59		
2	2	135	10.21		

ANALYSIS RESULTS

Structured

Raw Output

Step Log

Buy Monthly Membership

POLICY PARAMETERS USED

\$25/Day Other membership options Reduced Fare Bike Share Together, Citi Bike and Healthfirst are expanding access to bikeshare through \$5 monthly memberships for NYCHA residents and SNAP recipients.

\$4.99/ride Day pass When you've got lots to do.

Any ride longer than 30 minutes is an additional \$0.41/minute.

In return, they earn points toward ebike credits, free membership extensions, and more.

Starting at \$239/year or \$19.92/month Sign up Short-term options Short-term options Commitment is hard.

\$0.41/min for every minute thereafter.

COST COMPARISON

Pay-Per-Use Total

\$1617.00

Applies unlock fee + overage per ride

Monthly Membership

\$14.95

Fiat fee + any member surcharges

Savings

\$1437.60

With Buy Monthly Membership

Break-Even

5 rides

Per month to justify membership

Agent Timeline:



Result analysis:

JUSTIFICATION

** The total rides analyzed is 539. The pay-per-use total is \$1617.00. The monthly membership costs \$14.95 per month, which translates to \$179.40 per year. The break-even point is approximately 5 rides per month. Since the rider has already exceeded this threshold, buying a monthly membership would be the more cost-effective option.

WEEKLY BREAKDOWN

WEEK	RIDES	AVG DURATION (MIN)	EBIKE %	SPEND (PAY-PER-USE \$)
1	135	19.59		
2	135	10.21		
3	135	6.70		
4	134	9.04		

ASSUMPTIONS & CAVEATS

- & CAVEATS:**
- The pricing information is accurate as of the captured date (2026-03-24 06:47 UTC).
- The rider's usage pattern will remain consistent.
- The calculation of break-even rides is based on the monthly membership cost and the pay-per-use cost per ride.

CITATIONS

"Single Ride Day Pass Citi Bike Lyft Pink \$4.99 for 30 mins \$25/day \$239/year \$199/year Get the app Get a day pass Join Join Single Ride \$4.99 for 30 mins Get the app Bike unlocks \$4.99 free free free Ebike prices * \$0.41/min \$0.41/min \$0.27/min \$0.27/min Ride times 30 min 30 min 45 min 45 min Bike Angels Grubhub+ Rideshare benefits Day Pass \$25/day Get a day pass Bike unlocks \$4.99 free free free Ebike prices * \$0.41/min \$0.41/min \$0.27/min \$0.27/min Ride times 30 min 30 min 45 min 45 min Bike Angels Grubhub+ Rideshare benefits Citi Bike \$239/year Join Bike unlocks \$4.99 free free free Ebike prices * \$0.41/min \$0.41/min \$0.27/min \$0.27/min Ride times 30 min 30 min 45 min 45 min Bike Angels Grubhub+ Rideshare benefits Lyft Pink \$199/year Join Bike unlocks \$4.99 free free free Ebike prices * \$0.41/min \$0.41/min \$0.27/min \$0.27/min Ride times 30 min 30 min 45 min 45 min Bike Angels Grubhub+ Rideshare benefits *Prices subject to change."
Source: <https://citibikenyc.com/pricing> · Captured: 2026-03-24 06:47 UTC

"Citi Bike: Member-only perks Citi Bike: Member-only perks For less than a dollar a day, you get unlimited 45-minute rides on classic bikes, reduced ebike prices, free unlocks, and a whole lot more."
Source: <https://citibikenyc.com/pricing> · Captured: 2026-03-24 06:47 UTC

"\$25/Day Other membership options Reduced Fare Bike Share Together, Citi Bike and Healthfirst are expanding access to bikeshare through \$5 monthly memberships for NYCHA residents and SNAP recipients."
Source: <https://citibikenyc.com/pricing> · Captured: 2026-03-24 06:47 UTC

"\$4.99/ride Day pass When you've got lots to do."
Source: <https://citibikenyc.com/pricing> · Captured: 2026-03-24 06:47 UTC

"Any ride longer than 30 minutes is an additional \$0.41/minute."
Source: <https://citibikenyc.com/pricing> · Captured: 2026-03-24 06:47 UTC

"In return, they earn points toward ebike credits, free membership extensions, and more."
Source: <https://citibikenyc.com/pricing> · Captured: 2026-03-24 06:47 UTC

"Starting at \$239/year or \$19.92/month Sign up Short-term options Short-term options Commitment is hard."
Source: <https://citibikenyc.com/pricing> · Captured: 2026-03-24 06:47 UTC

"\$0.41/min for every minute thereafter."
Source: <https://citibikenyc.com/pricing> · Captured: 2026-03-24 06:47 UTC

Pre-step logs:

Structured Raw Output Step Log

Structured Raw Output Step Log

Raw output:

Structured Raw Output Step Log

****DECISION:** Buy Monthly Membership**

****JUSTIFICATION:****

The total rides analyzed is 539. The pay-per-use total is \$1617.00. The monthly membership costs \$14.95 per month, which translates to \$179.40 per year. The break-even point is approximately 5 rides per month. Since the rider has already exceeded this threshold, buying a monthly membership would be the more cost-effective option.

COST BREAKDOWN:

- Total rides analysed: 539
- Pay-per-use total: \$1617.00
- Monthly membership: \$14.95
- Break-even rides: 5 rides/month (actual: 539)
- Savings: \$1437.60 with Monthly Membership

****WEEKLY TABLE:****

Week	Rides	Avg Duration (min)	eBike %	Spend (pay-per-use \$)
1	135	19.59		
2	135	10.21		
3	135	6.70		
4	134	9.04		

2. Scenario where pay-per-user wins:

CONFIGURE ANALYSIS

Monthly Trip CSV

Drop CSV here or click to browse

JC-SS202602-citibike-tripdata.csv

Pricing Page URL

<https://citibikenyc.com/pricing>

LLM Model

Llama 3.3 70B (Groq)

▶ Run Analysis

RUN METRICS

2

STEPS

JC-S202602-citibike-tripdata.csv

3.14s

TOTAL TIME

final answer

STOP REASON

AGENT TIMELINE

WEEKLY TABLE

Week	Rides	Avg Duration (min)	eBike %	Spend (pay-per-use \$)
1	1	4.0	50%	\$4.99
2	1	3.0	0%	\$4.99

ASSUMPTIONS & CAVEATS:

- The analysis assumes that the rider will not exceed the 30-minute ride time limit.

- The analysis does not take into account any potential discounts or promotions that may be available.

- The analysis is based on the current pricing policy and may not reflect any future changes.

CITATIONS:

- "Single Ride: \$4.99 for 30 mins" — source: <https://citibikenyc.com/pricing>, captured: 2026-03-24 06:56 UTC

- "Any ride longer than 30 minutes is an additional \$0.41/minute" — source: <https://citibikenyc.com/pricing>, captured: 2026-03-24 06:56 UTC

- "Membership: \$239/year or \$19.92/month" — source: <https://citibikenyc.com/pricing>, captured: 2026-03-24 06:56 UTC

ANALYSIS RESULTS

Structured

Raw Output

Step Log

Pay Per Ride / Minute

POLICY PARAMETERS USED

\$25/Day Other membership options Reduced Fare Bike Share Together, Citi Bike and Healthfirst are expanding access to bikeshare through \$5 monthly memberships for NYCHA residents and SNAP recipients.

In return, they earn points toward ebike credits, free membership extensions, and more.

\$4.99/ride Day pass When you've got lots to do.

Any ride longer than 30 minutes is an additional \$0.41/minute.

Starting at \$239/year or \$19.92/month Sign up Short-term options Short-term options Commitment is hard.

COST COMPARISON

Pay-Per-Use Total

\$9.98

Applies unlock fee + overage per ride

Monthly Membership

\$19.92

Flat fee + any member surcharges

Savings

\$9.94

With Pay Per Ride / Minute

Break-Even

4 rides

Per month to justify membership

JUSTIFICATION

** The total number of rides is 2, which is a relatively low number. The pricing policy shows that the cost of a single ride is \$4.99 for